pfi

Python for Fixed Income

COURSE 7 · NLP, RAG, AND LLMS

Sentence Embeddings and Semantic Search

WEEK 2 · DAY 1

Created by **Jeff Lenamon**

This course is educational content created in a personal capacity. Nothing here is investment advice or a recommendation to buy or sell any security. All portfolio data is synthetic.

From word vectors to a sentence model

Day 4: one vector per word

Looked up in a table, the same in every sentence; a document is the mean of its words.

Today: one vector per passage

Computed from all its words in their order. A question becomes a vector too.

22,713,216

parameters, MiniLM

384

numbers per text

256

word pieces kept per text

Real documents, described

A search result on the indenture: a section, a score, and its own words

Allowed names	Never called
the indenture states	loose, tight, weak, strong
extracted value, with its quote	covenant-lite or protective
passed the source check	a credit view or recommendation

A score says how close two texts sit in the model's numbers, not what the indenture means.

Sections into chunks

107

sections

454

chunks, 348 of the full 180 words

997

words before 1.01, in no chunk

```
chunks = textkit.chunk_text(sections, indenture)   # 180 words, the next 150 later
# Section 4.12: 1,819 words, 12 chunks, the last 169 words, clause (e) kept
```

Most chunks: 1.01 113, 12.14 72, 2.05 30. Section 12.14's run on through the signatures and the exhibits.

Source: Boyd Gaming Corporation, indenture for the 4.750% Senior Notes due 2031, Exhibit 4.1 to the 8-K filed 2021-06-08, accession 0001193125-21-185495.

How a sentence model works

The problem	Passages with similar meaning should get similar lists of numbers
Pieces	The tokenizer cuts the text; each piece starts as a looked-up vector
Six layers	In each, every piece reads from every other and updates its own vector
The average	The pieces' vectors are averaged into 384 numbers, scaled to length 1
What you control	Which model, which passage; out comes one row of 384 numbers per text

A sentence model, and where its files went

11

files in the Hugging Face cache

91,578,415

bytes on disk

1

download, then no network

```
~\.cache\huggingface\hub          # on Windows; HF_HOME moves it
hf cache ls                           # each model and its size
hf cache rm model/sentence-transformers/all-MiniLM-L6-v2
```

The first load, download included, took about 7 seconds on the build machine.

Where sentence embeddings come from

The paper	Sentence-BERT, Reimers and Gurevych: arXiv 1908.10084, 2019-08-27, EMNLP 2019
Its abstract	"semantically meaningful sentence embeddings that can be compared using cosine-similarity"
Model card	"input text longer than 256 word pieces is truncated"

Both statements are measured on the indenture today. The paper itself was not read.

Sources, read 8 October 2026: arxiv.org/abs/1908.10084; huggingface.co/sentence-transformers/all-MiniLM-L6-v2 (pinned revision)

How embed works

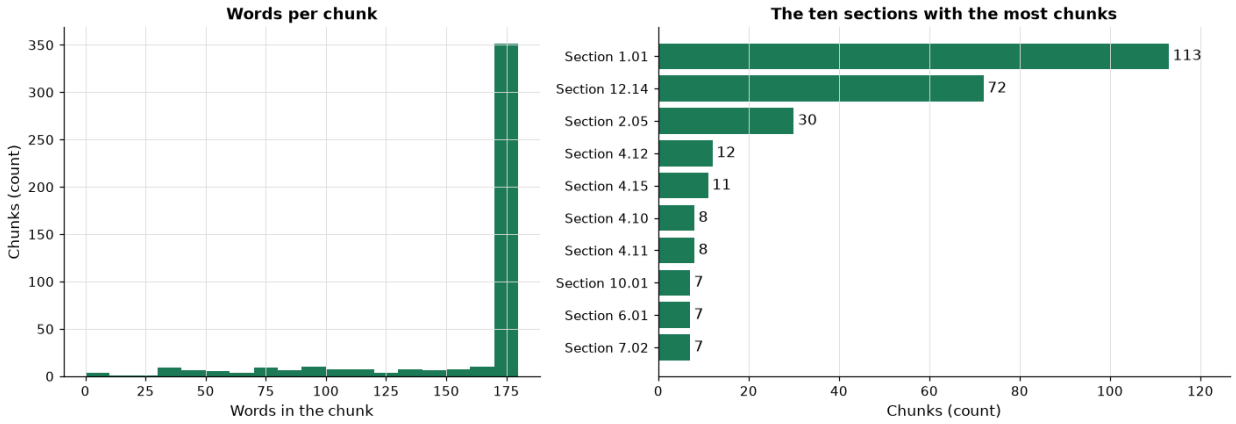
The problem	Many texts in, one array of vectors out, in the order given
Batches	encode sorts the texts by length, cuts them into batches of 64, runs each through the model
Scaling	normalize_embeddings=True: every row has length 1, so a dot product is a cosine
What you control	Batch size, normalization, the progress bar; section 6.11 lists all 15 settings
What comes out	A float32 array with one row per text

Embeddings, normalized, embedded once

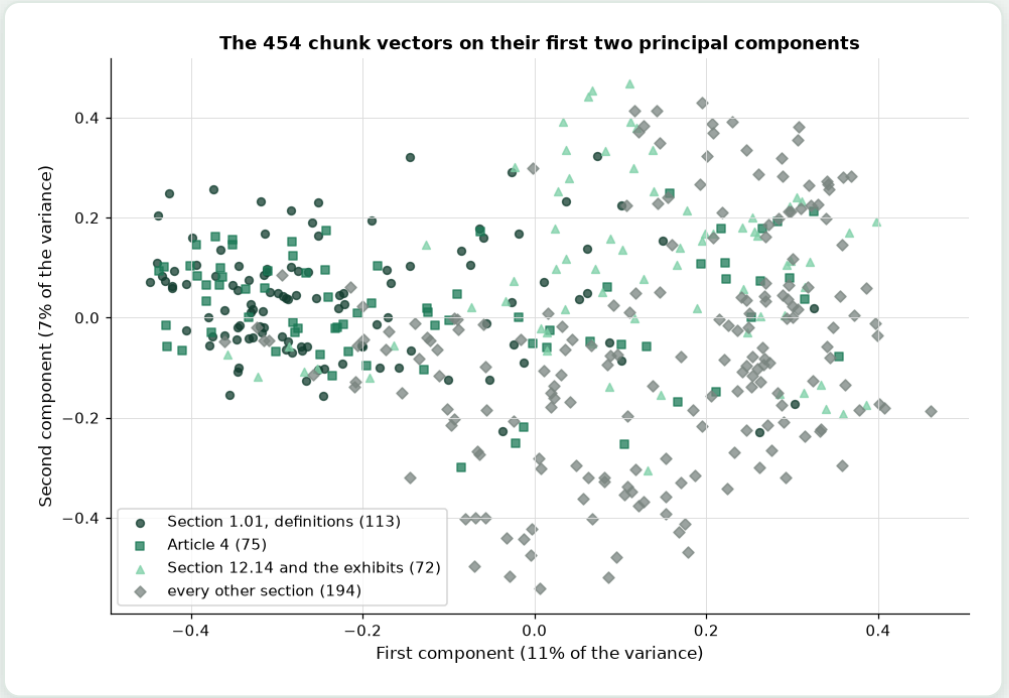
```
matrix = textkit.embed(model, chunks["text"].tolist())
# 454 rows of 384 numbers, 0.70 MB; chunk 0 starts -0.0977, 0.0184, -0.0135
# dot product equal to the cosine within 1e-6 on all 206,116 pairs
# 8 chunks embedded alone: 0 of 8 rows bit-identical, all within 1e-6
```

Embed a list once, reuse the array, compare within 1e-6, print scores at 4 decimals.

The chunks and their vectors, drawn



348 of the 454 chunks hold the full 180 words; Sections 1.01 and 12.14 hold most. The two components keep only part of the variance: a rough guide.



How a semantic search works

The problem	Find the chunks closest in meaning to a question, with no shared word needed
Embed	The question goes through the same model, so it sits in the same space
Score	Each chunk's dot product with the question: for unit rows, the cosine
top_k	One matrix product, a sort, the k largest rows, ties to the lower row
What comes out	Rows and scores: places to read, not answers

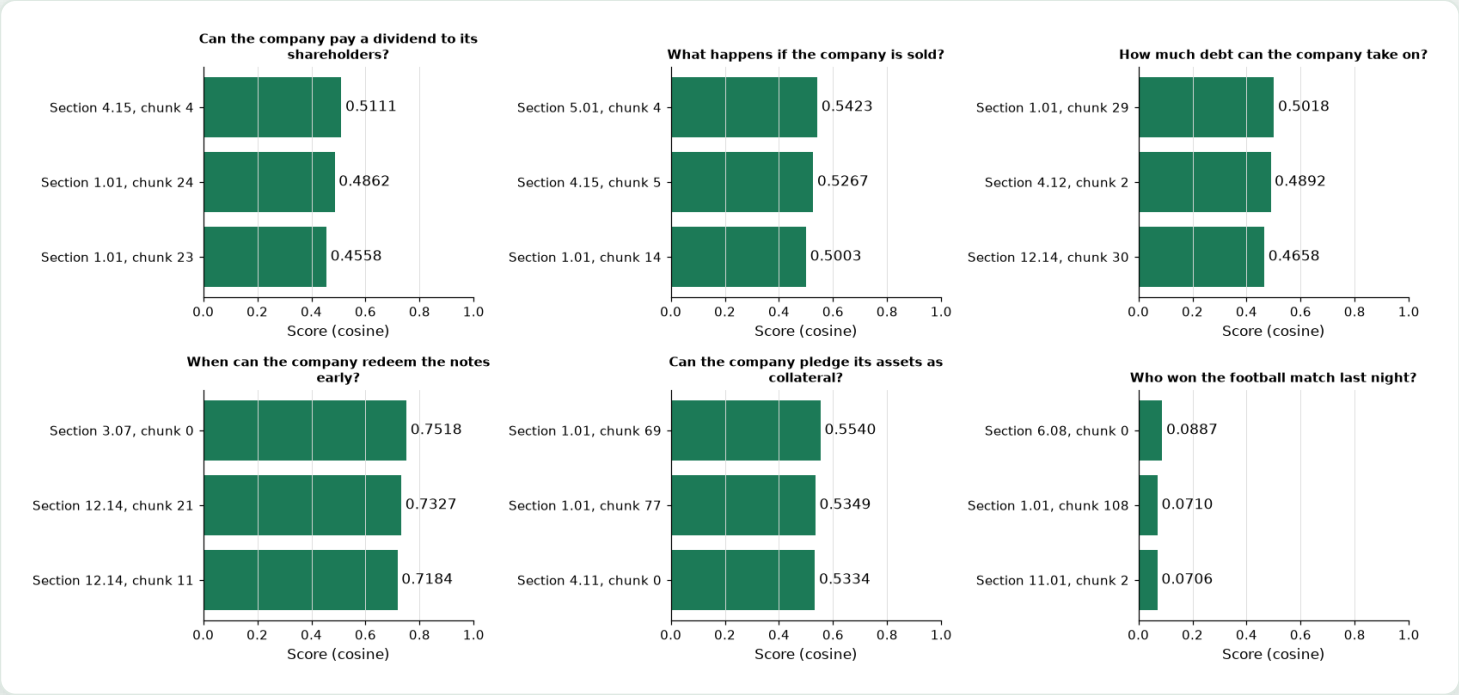
Semantic search

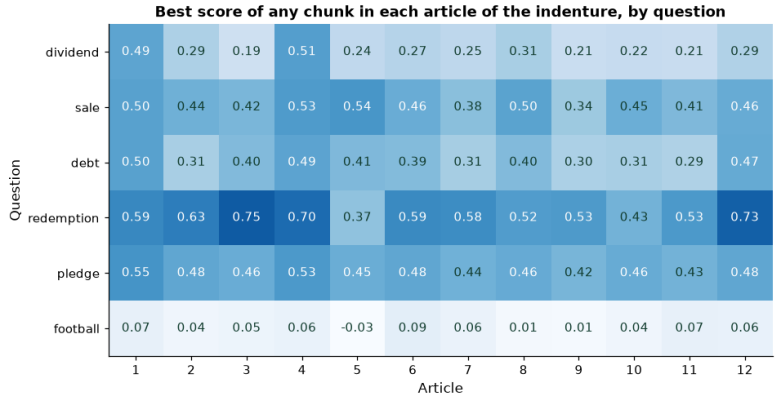
Question	Top section	Title	Score
Pay a dividend?	4.15	Limitation on Restricted Payments	0.5111
The company is sold?	5.01	Merger, Consolidation and Sale of Assets	0.5423
How much debt?	1.01	Definitions	0.5018
Redeem the notes early?	3.07	Optional Redemption	0.7518
Pledge assets as collateral?	1.01	Definitions	0.5540

A search ranks chunks. Reading the section says what it answers.

Source: Boyd Gaming Corporation, indenture for the 4.750% Senior Notes due 2031, Exhibit 4.1 to the 8-K filed 2021-06-08, accession 0001193125-21-185495.

Search, drawn





Each row's darkest cell is where the question lands. The football row is pale throughout.

Source: Boyd Gaming Corporation, indenture for the 4.750% Senior Notes due 2031, Exhibit 4.1 to the 8-K filed 2021-06-08, accession 0001193125-21-185495.

How a word piece tokenizer works

The problem	A model needs numbers, and any text must be readable, even a word it never saw
The list	A fixed list of 30,522 pieces, made before the model was trained
The cut	Lower-case, split at spaces and punctuation, longest piece that fits from the left
Special pieces	[CLS] at the start, [SEP] at the end; every piece has an id, its row in the list
What comes out	Pieces and ids; the model keeps the first 256

Subword tokens and the 256 limit

Word	Word pieces
Indebtedness	ind, ##eb, ##ted, ##ness
EBITDA	e, ##bit, ##da
Consolidated	consolidated
Restricted	restricted

34

of 454 chunks over 256 pieces

316

pieces, the longest chunk

140

words kept, the fewest

Median 224 pieces a chunk; 50 of 107 whole sections over 256 (4.12 alone 2,561).

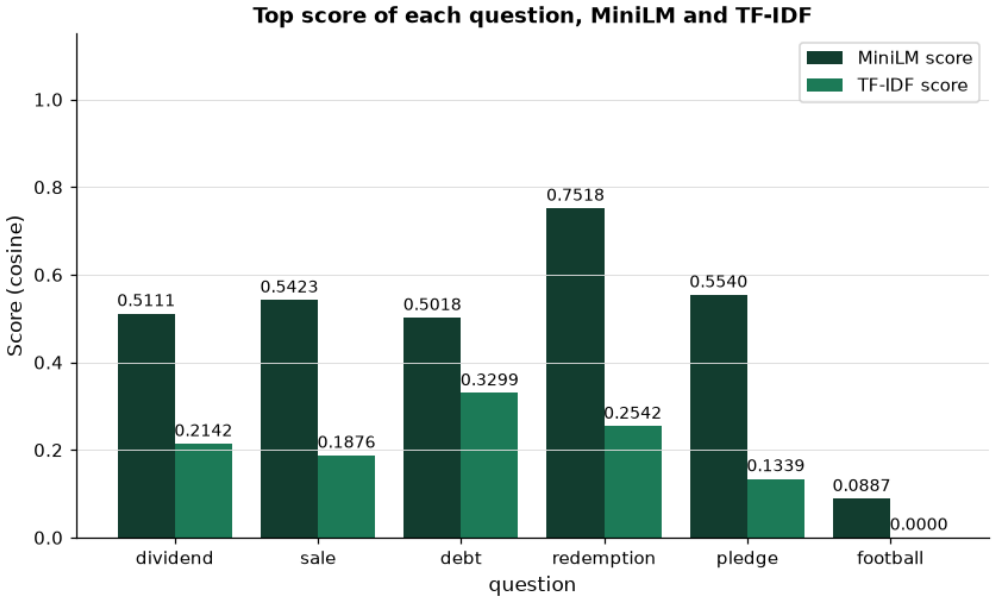
A sentence added past piece 256: identical vector (True). The model cuts silently.

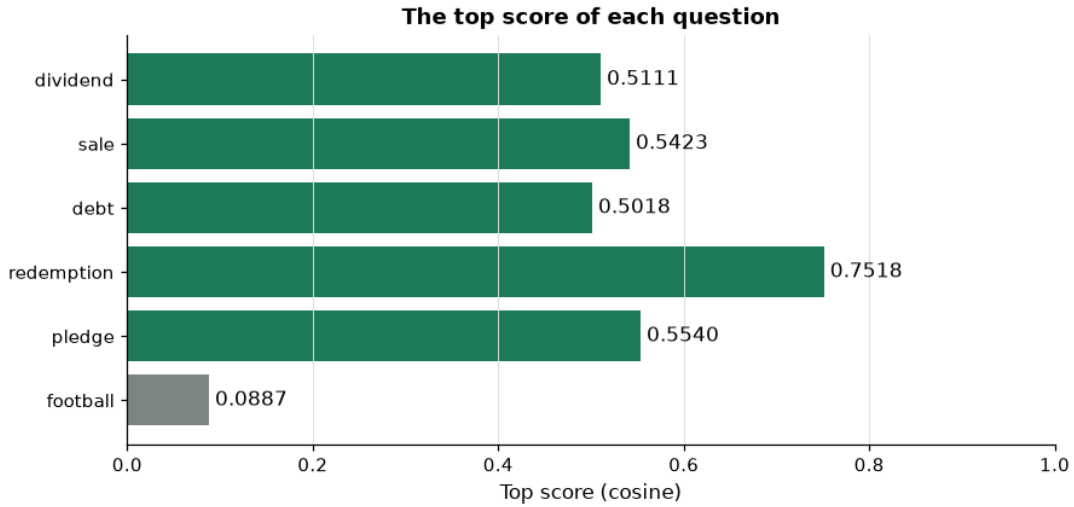
The same questions by TF-IDF

Question	MiniLM	Score	TF-IDF	Score
Pay a dividend?	4.15	0.5111	4.16	0.2142
The company is sold?	5.01	0.5423	2.01	0.1876
How much debt?	1.01	0.5018	12.09	0.3299
Redeem the notes early?	3.07	0.7518	3.07	0.2542
Pledge assets as collateral?	1.01	0.5540	1.01	0.1339
Football match (control)	6.08	0.0887	1.01	0.0000

Word overlap needs the document's own words; 2,915 columns, stop words out.

Two methods, and the six scores





The scales differ: compare the sections, not the bars. A score ranks the chunks for one question.

A score is not a probability

0.7518

redemption, Section 3.07

0.5111

dividend, Section 4.15

0.0887

football, Section 6.08

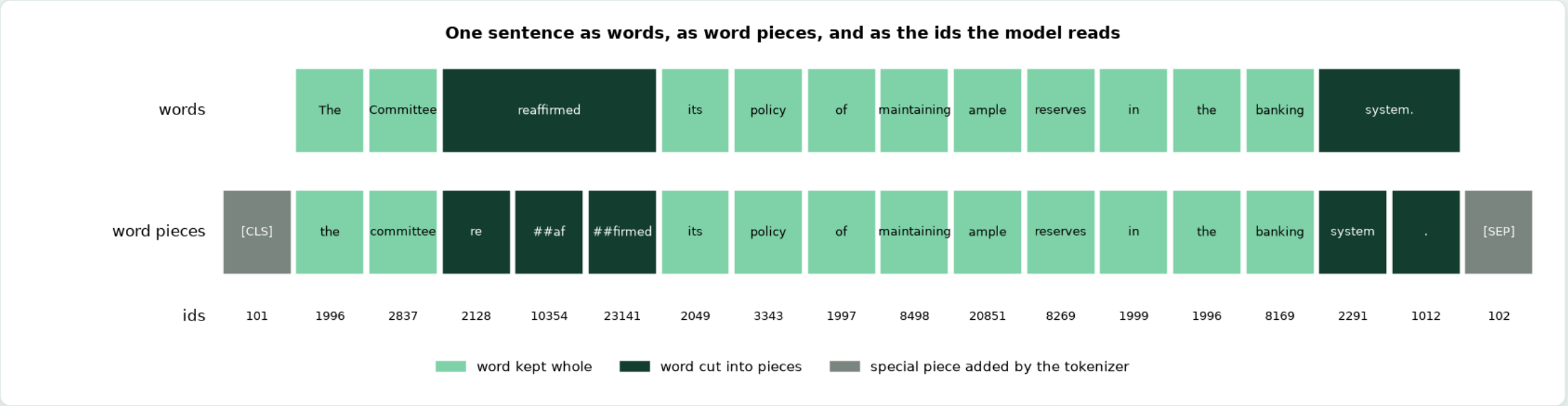
A cosine between -1 and 1. Not a probability, not a confidence; it ranks chunks for one question and does not compare questions. top_k always returns k rows.

A search gives places to read. The reading stays with you.

How attention works

The problem	A piece's vector should depend on the pieces around it
Query and key	Each piece forms a query, a key, and a value from its own vector
Weights	A query against every key gives scores, turned into weights that add up to 1
The mix	The new vector is the weighted mix of the values: 12 heads, 6 layers
What you read	A table of weights per head: where it reads from, not why

A sentence as words, pieces, and ids



13

words

16

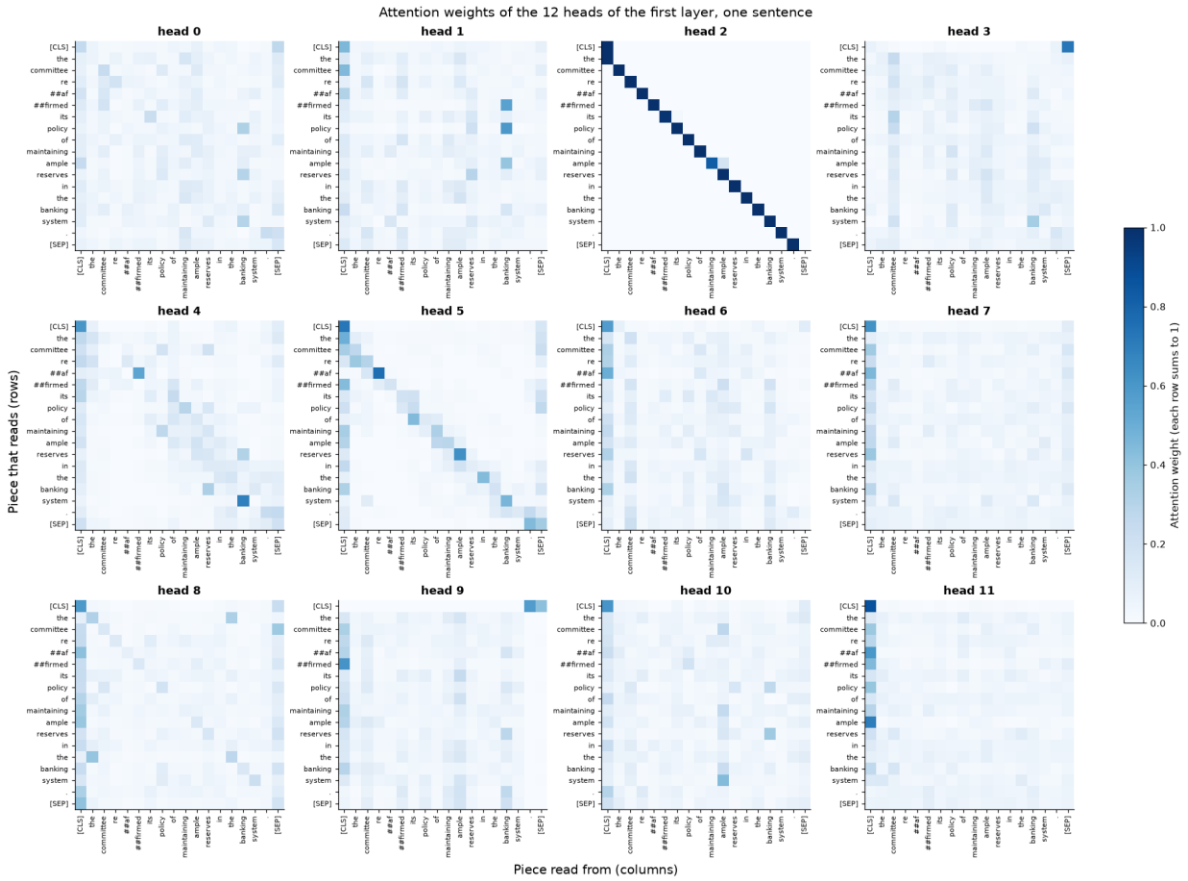
word pieces

18

ids

The model never reads letters, only the ids. The same piece has the same id wherever it stands.

Attention, head by head



0.99

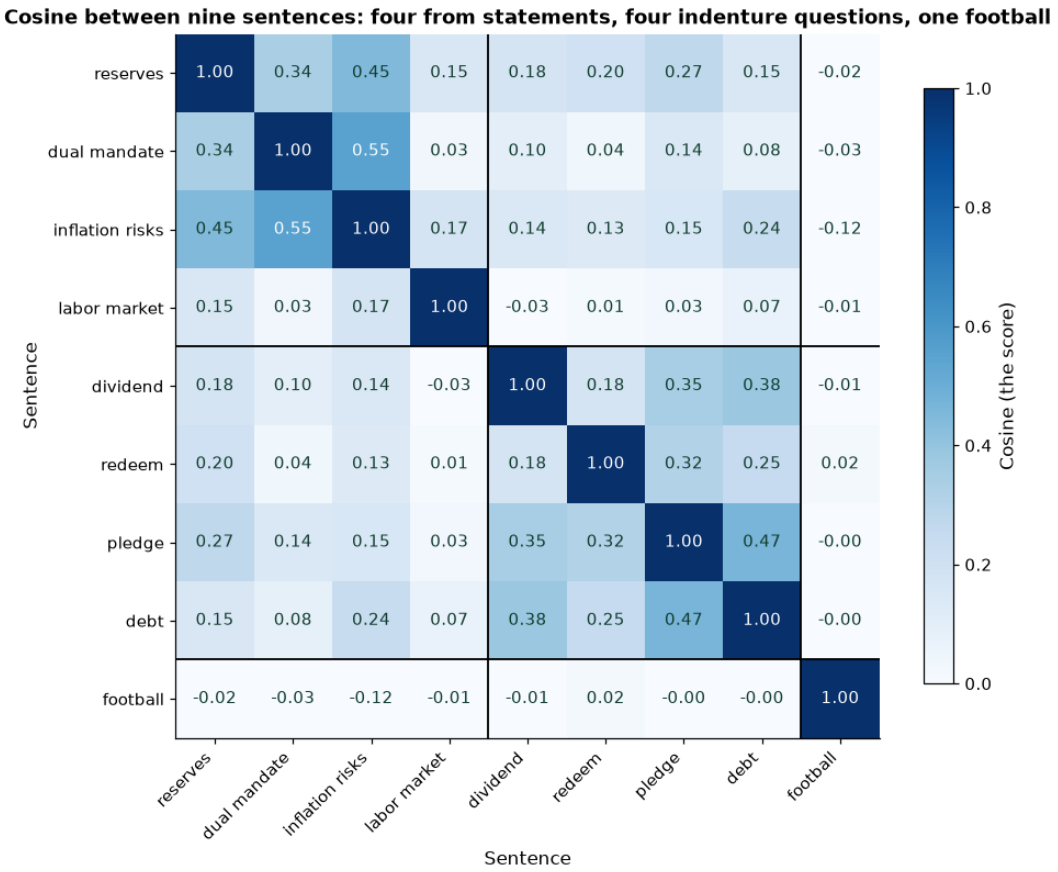
of head 2's weight on the piece before

0.33

head 5, the next most

Share of the weight on [CLS], averaged over the 12 heads, in layers 1 to 6: 0.19, 0.56, 0.56, 0.52, 0.57, 0.31. A weight shows where a head reads from, not why.

Sentences against sentences



0.28

inside the 4 statement sentences

0.32

inside the 4 questions

0.12

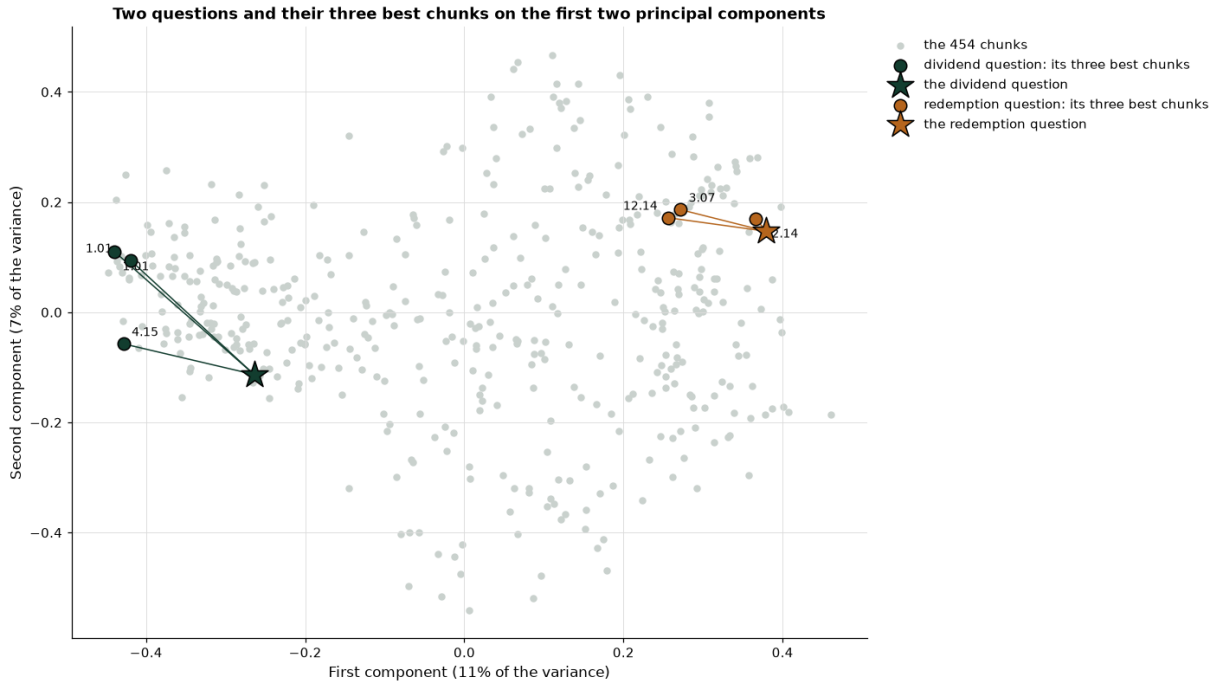
between the two groups

-0.02

football with the other eight

The closest pairs share a subject, risks or borrowing. The labor market sentence, on a subject nothing else shares, is close to none.

A search, drawn



Among the 3 nearest on the page: 0 of the dividend question's 3 best chunks, 1 of the redemption question's. The search ranks with 384 numbers; the page only shows where the results sit.

How the settings of encode work

The problem	Fifteen settings: which of them change the vectors?
What is embedded	A prefix put in front of each text
Speed	Batch size, device, progress bar: the last digits at most
What comes back	The type of the numbers, how many are kept, scaled to length 1 or not
Several processes	A pool of processes and the size of the pieces sent to each

Every setting of encode

prompt_name (default None)

The name of a prompt the model stores; its text goes in front of every input.

prompt (default None)

Text put in front of every input.

output_value (default 'sentence_embedding')

One vector per text ('sentence_embedding') or one per word piece ('token_embeddings').

precision (default 'float32')

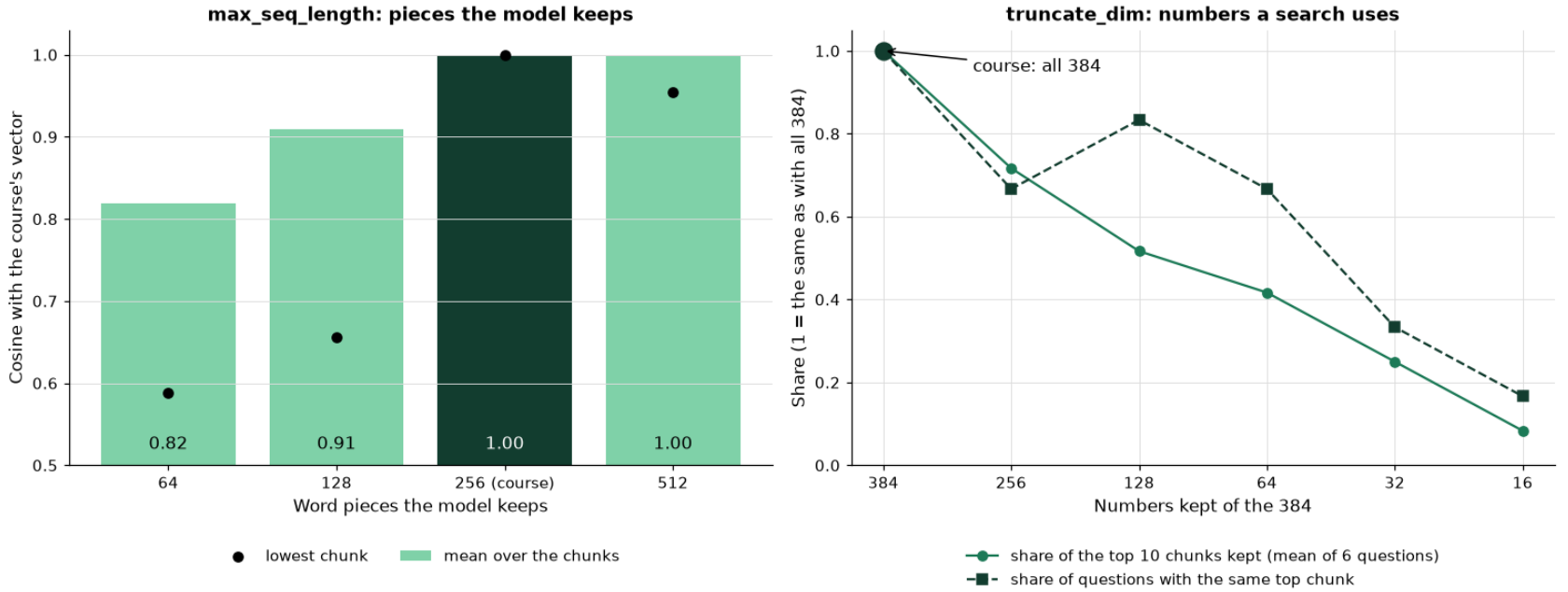
The type of the numbers: float32, int8, uint8, binary, or ubinary.

truncate_dim (default None)

Keep only the first k of the 384 numbers.

15 settings, read from the installed library; the five shaded in the notebook can change what the vectors mean. The course sets 4: batch_size=64, show_progress_bar=False, normalize_embeddings=True, convert_to_numpy=True.

What two settings change



Cosine with the 256-piece vectors: 64 pieces 0.82 (lowest 0.59); 128 pieces 0.91 (lowest 0.66); 512 pieces 1.00 (lowest 0.95). Share of the top 10 kept: 384 1.00; 256 0.72; 128 0.52; 64 0.42; 32 0.25; 16 0.08. Six questions: read the shape, not the digits.

Excel side by side

Excel side by side (tested in Excel)

`=SUMPRODUCT(A1:A384,B1:B384)`	0.5111: top_k's score, within 1e-12
`=SUMPRODUCT(A1:A384,B1:B384)/(SQRT(SUMSQ(A1:A384))*SQRT(SUMSQ(B1:B384)))`	0.5111, within 1e-12
`=SQRT(SUMSQ(A1:A384))`	1.0000: length 1 within 1e-6
`=ROWS(A1:A384)`	384

No tokenizer or sentence model in Excel: the 384 numbers come from MiniLM.

Save the day: show the commit

```
git show --stat HEAD
test_textkit.py |  69 ++++++++++++++++++++++++++++
textkit.py      | 134 ++++++++++++++++++++++++++++
```

The AI check

The docstring is the prompt. The code that comes back runs, and its number looks fine.

1

Docstring

Units, conventions, and what the function must never do: what you asked for.

→

2

Code back

Written for the course in the style of an AI answer. Read it.

→

3

Test first

Assert what the docstring promises. Run it: it fails.

→

4

Fix

Find the rule it broke, fix it, run the test again.

Today: the loop's range drops the last partial chunk: 347 chunks, 53 sections with none, 4.12's clause (e) lost.

Code that runs is not the same as code that is right.

What you can do now

- Chunk sections, never across a boundary
- Load a pinned model; know its cache
- Embed once; dot product equals cosine
- Search by meaning; report section, score, words
- Word pieces, ids, the 256 limit, attention

Week 2, Day 2 Zero-shot classification against TF-IDF